EE-170L Computer Programming Lab

Laboratory Report — No. 13

Structure in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	13
Topic	struct Declaration, Members, Dot Notation, Nested Structs
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

- Understand a structure as a user-defined type that groups variables of different data types.
- Declare a struct with multiple members of mixed types (int, float, char, char[]).
- Define structure variables and access members using the dot (.) operator.
- Initialize structure variables at declaration using curly-brace syntax.
- Copy one structure variable to another and verify member values are duplicated.
- Apply structures to model real-world entities: persons, students, and books.

2. Theoretical Background

2.1 What is a Structure?

A structure is a collection of variables grouped under a single name. Unlike arrays, structure members can have different data types — some int, some float, some char, and so on. The data items inside a structure are called its members.

- A structure is defined using the struct keyword followed by a tag (name) and a block of member declarations.
- The closing brace of a struct definition must be followed by a semicolon.
- A struct definition only defines a blueprint — it does not reserve memory until a variable of that type is declared.
- One structure can be nested inside another structure.
- Structure variables can be passed to functions by value or by reference.

2.2 Declaring a Structure

Syntax:

```
struct tagName
{
    dataType member1;   // first member
    dataType member2;   // second member
    // ... more members
```

```
};                          // semicolon required after closing brace
```

Example — declaring a structure called part:

```
struct part
{
    int   modelnumber;  // ID number of the model
    int   partnumber;   // ID number of the part
    float cost;         // cost of the part in dollars
};
```

2.3 Declaring Structure Variables and Accessing Members

After defining the structure, declare a variable of that type and access its members using the dot (.) operator:

```
part part1;               // declare a structure variable of type part

// Assign values to members using dot notation
part1.modelnumber = 6244; // access member modelnumber
part1.partnumber  = 373;  // access member partnumber
part1.cost        = 217.55F; // access member cost

// Read member values
cout << part1.modelnumber << endl;
```

2.4 Initialization at Declaration and Copying Structs

A structure variable can be initialized at declaration using curly braces. One structure variable can also be copied to another with a simple assignment:

```
part part1 = {6244, 373, 217.55F}; // initialize all members at once
part part2;                         // declare second variable
part2 = part1;                      // copy all members from part1 to part2
```

3. Example Programs

3.1 Structure with One Variable

```
#include <iostream>
using namespace std;

struct part              // define the structure blueprint
{
    int   modelnumber;   // stores the model ID
    int   partnumber;    // stores the part ID
    float cost;          // stores the cost in dollars
};

int main()
{
    part part1;          // declare a variable of type part (memory reserved
here)

    // Assign values to each member via dot notation
    part1.modelnumber = 6244;
    part1.partnumber  = 373;
    part1.cost        = 217.55F;

    // Display all member values
    cout << "Model : " << part1.modelnumber << endl;
    cout << "Part  : " << part1.partnumber  << endl;
    cout << "Cost  : " << part1.cost        << " $" << endl;
```

```
    return 0;
}
```

Output:

```
Model : 6244
Part  : 373
Cost  : 217.55 $
```

3.2 Structure with User Input — Person

```
#include <iostream>
using namespace std;

struct Person            // structure to represent a person
{
    char  name[50];      // name stored as C-string (up to 49 characters)
    int   age;           // age in years
    float salary;        // monthly salary in currency units
};

int main()
{
    Person p1;           // declare one Person variable

    // Read full name — cin.get() captures spaces
    cout << "Enter Full name: ";
    cin.get(p1.name, 50);

    cout << "Enter age: ";
    cin >> p1.age;

    cout << "Enter salary: ";
    cin >> p1.salary;

    // Display all collected information
    cout << "\nDisplaying Information." << endl;
    cout << "Name   : " << p1.name   << endl;
    cout << "Age    : " << p1.age    << endl;
    cout << "Salary : " << p1.salary << endl;

    return 0;
}
```

Output:

```
Enter Full name: Murad Ali
Enter age: 34
Enter salary: 1200

Displaying Information.
Name   : Murad Ali
Age    : 34
Salary : 1200
```

4. Lab Tasks & Solutions

Task 1 — Structure: Person

Declare a structure 'Person' with members: name (string), age (integer), address (string). Declare a variable 'person1', initialize its members, and display them using dot notation.

```
#include <iostream>
using namespace std;

// Define the Person structure with three members
struct Person
{
    char name[50];       // person's full name as a C-string
    int  age;            // person's age in years
    char address[100];   // person's home address as a C-string
};

int main()
{
    // Declare a variable of type Person
    Person person1;

    // Initialize each member using the dot operator
    // strcpy is needed because C-strings cannot be assigned with =
    // after declaration — instead, we initialize directly in this style:
    Person p = {"Zawar Shah", 20, "Nowshera, KPK, Pakistan"};
    person1 = p;         // copy initialized struct to person1

    // Display all member values using dot notation
    cout << "---- Person Information ----"        << endl;
    cout << "Name    : " << person1.name    << endl;
    cout << "Age     : " << person1.age     << endl;
    cout << "Address : " << person1.address << endl;

    return 0;
}
```

Output:

```
---- Person Information ----
Name    : Zawar Shah
Age     : 20
Address : Nowshera, KPK, Pakistan
```

How it works:

- The struct keyword defines the Person blueprint with three members of different types.
- Person person1; declares a variable, which allocates memory for all three members.
- The dot operator (.) accesses individual members: person1.name, person1.age, person1.address.

Task 2 — Structure: Student

Declare a structure 'Student' with members: name (string), age (integer), grade (char). Declare variable 'student1', initialize, and display its members.

```
#include <iostream>
using namespace std;

// Define the Student structure with academic fields
struct Student
{
    char name[50];       // student's full name
    int  age;            // student's age
    char grade;          // academic grade: A, B, C, D, or F
};

int main()
{
```

```
    // Declare and initialize student1 using curly-brace syntax
    Student student1 = {"Zawar Shah", 20, 'A'};  // all members set at once

    // Display each member of student1 using the dot operator
    cout << "---- Student Record ----" << endl;
    cout << "Name  : " << student1.name  << endl; // display char array
    cout << "Age   : " << student1.age   << endl; // display integer
    cout << "Grade : " << student1.grade << endl; // display single character

    // Modify a member after initialization and re-display
    student1.grade = 'B';  // update grade through dot notation
    cout << "\nAfter grade update:"            << endl;
    cout << "Grade : " << student1.grade << endl; // should print B

    return 0;
}
```

Output:

```
---- Student Record ----
Name  : Zawar Shah
Age   : 20
Grade : A

After grade update:
Grade : B
```

How it works:

- Curly-brace initialization assigns all members in the order they appear in the struct definition.
- grade is of type char, so single quotes ('A') are used — not double quotes.
- Members can be modified at any time after declaration using dot notation.

Task 3 — Accessing Structure Elements: Book

Declare a structure 'Book' with members: title (string), author (string), price (float), pages (integer). Declare two variables book1 and book2, initialize both, and display their members.

```
#include <iostream>
using namespace std;

// Define the Book structure to represent a book record
struct Book
{
    char  title[100];    // book title (C-string)
    char  author[50];    // author's full name (C-string)
    float price;         // retail price
    int   pages;         // total number of pages
};

// Helper function to display all members of a Book variable
void displayBook(Book b)
{
    cout << "  Title  : " << b.title  << endl;
    cout << "  Author : " << b.author << endl;
    cout << "  Price  : $" << b.price  << endl;
    cout << "  Pages  : " << b.pages  << endl;
}

int main()
{
    // Declare and initialize book1 with values at declaration
```

```
    Book book1 = {"C++ Programming", "Bjarne Stroustrup", 1376.00F, 1376};

    // Declare book2 and assign members individually
    Book book2;
    // Assign using the book2 variable — for C-strings use struct init or strcpy
    Book b2 = {"The C Programming Language", "Kernighan & Ritchie", 950.00F,
272};
    book2 = b2;          // copy all members from b2 to book2

    // Display both book records
    cout << "=== Book 1 ===" << endl;
    displayBook(book1);

    cout << "\n=== Book 2 ===" << endl;
    displayBook(book2);

    return 0;
}
```

Output:

```
=== Book 1 ===
  Title  : C++ Programming
  Author : Bjarne Stroustrup
  Price  : $1376
  Pages  : 1376

=== Book 2 ===
  Title  : The C Programming Language
  Author : Kernighan & Ritchie
  Price  : $950
  Pages  : 272
```

How it works:

- book1 is fully initialized at declaration using curly braces.
- book2 is declared first, then an initialized temporary struct b2 is copied into it.
- displayBook() accepts a Book variable by value and prints each member using dot notation.
- Passing a struct to a function by value creates a complete copy — the original is unaffected.

5. Structure vs Array — Key Differences

Feature	Array	Structure (struct)
Element types	All same type	Different types allowed
Declaration	int a[5];	struct Tag { ... }; Tag var;
Member access	a[0], a[1], ...	var.member
Grouping purpose	Collection of same data	Grouping of related mixed data
Copying	Must copy element-by-element	var2 = var1; copies all members
Function passing	Array decays to pointer	Passed by value (full copy)

6. Conclusion

This lab introduced structures — a user-defined data type that groups related variables of different types under a single name. Key takeaways:

- **struct Keyword:** Defines a new data type blueprint. No memory is allocated until a variable of that type is declared.
- **Mixed Types:** Unlike arrays, struct members can be int, float, char, char[], or even other structs.
- **Dot Operator:** variable.member accesses any member of a struct variable for reading or writing.
- **Initialization:** Curly-brace initialization {v1, v2, ...} assigns all members in declaration order at once.
- **Struct Copying:** A simple assignment (var2 = var1) copies all members — unlike arrays, which must be copied element-by-element.

Structures are the foundation of object-oriented programming. Mastering struct prepares you directly for classes, constructors, and encapsulation covered in advanced C++ labs.